# How Much Prompt-Injection Defense Does a Free Tool Whitelist Buy? Zero-Inference Capability Manifests for Tool-Calling Agents

Clement Aidoo
CANLABS, Ghana clement.aidoo@canlabs.org
Code, data and protocol: https://github.com/CAN-Aidoo/capscope Archive: doi:10.5281/zenodo.23636157

***Abstract***

**Defenses against indirect prompt injection that work well today put a language model in the loop, which adds model calls and latency to every task. We ask how much protection a tool-calling agent can get for free. capscope builds a tool whitelist (a *capability manifest*) once from the user's instruction by matching stemmed action verbs against per-tool profiles written from documentation. It then gates every tool call, and nothing the agent reads later can widen the whitelist. Building the manifest takes under a millisecond and no model call. In an oracle evaluation on all 949 user×injection pairs of AgentDojo v1.2, using the benchmark's ground-truth tool calls, the manifest keeps every needed tool for 91% of benign tasks and denies a tool the attack needs in 77% of pairs (95% interval 70–82%). Mechanically extracted keywords do much worse (56%). Even an oracle whitelist that knows exactly which tools each user task needs blocks only 87%. The other 13% of attacks, 33% in banking, use only tools the user's own task needs, so no tool-level policy can stop them. Most benign failures come from tasks that delegate their instructions to an email, file or web page, which is the channel injections use. A free whitelist therefore removes most attacks that need a new capability, and the 13% that reuse the user's own capabilities mark where argument- and data-flow-level defenses become necessary. An end-to-end evaluation with a model remains to be done; protocol, profiles, code (including the model-run harness) and per-pair results are public.**

## I. Introduction

Tool-calling language-model agents read untrusted text: emails, web pages, shared files, chat messages. An attacker who controls that text can embed instructions, and the agent may follow them with the user's privileges [1, 2]. This *indirect prompt injection* is the top risk in OWASP's list for LLM applications [3]. Defenses that hold up well today, such as CaMeL [5], Progent [6] and MELON [7], put a language model or a policy synthesiser in the loop. That costs extra model calls, latency and money on every task.

For many deployments that cost matters. At CANLABS we build agents for low-bandwidth, metered settings, where an extra round trip to a model is a real price. This raises a simple question: how much protection is available *for free*? The cheapest least-privilege policy we can think of is a tool whitelist built once from the user's instruction, by keyword matching, before the agent reads anything untrusted. Nothing the agent reads later can widen it. It needs no model call and runs in under a millisecond.

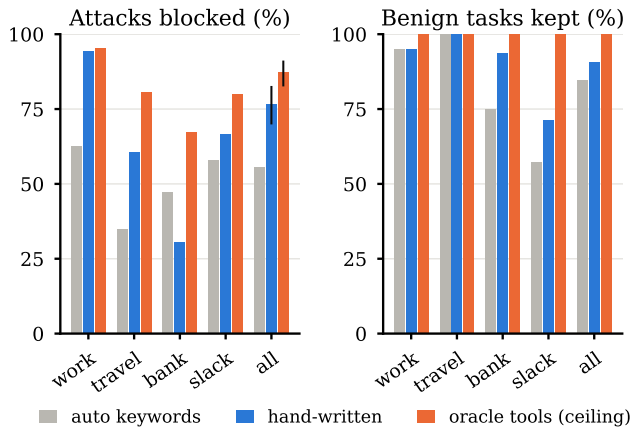



**Figure 1.** Oracle evaluation on AgentDojo v1.2 (949 user×injection pairs). Left: pairs in which the manifest built from the user's instruction denies at least one tool the injection needs. Right: benign user tasks whose every ground-truth call stays allowed. The orange "oracle tools" manifest knows exactly which tools the user task needs; it bounds every tool-level policy, so the 13% it cannot block are out of reach of any tool whitelist. Bars on "all": 95% bootstrap intervals over user tasks.

We call this whitelist a *capability manifest* and measure both what it buys and where any tool whitelist must stop. Our contributions are:

- **capscope**, a zero-inference capability manifest for AgentDojo agents [4]: per-tool profiles (category, sensitivity, action verbs) written from tool documentation alone, a deterministic matcher, and a gate that replaces the tool executor.
- An *oracle* evaluation that needs no model: on all 949 user×injection pairs of AgentDojo v1.2 we check the manifest against the ground-truth tool calls of the user task and of the injection. The hand-written manifest keeps 91% of benign tasks fully allowed and denies a needed tool in 77% of attacks.
- A ceiling for the whole class of tool-level defenses: even a manifest that knows exactly which tools the user task needs blocks only 87% of pairs. The remaining 13% (33% in banking) reuse tools the user's own task needs. Stopping them requires argument-level or data-flow control.

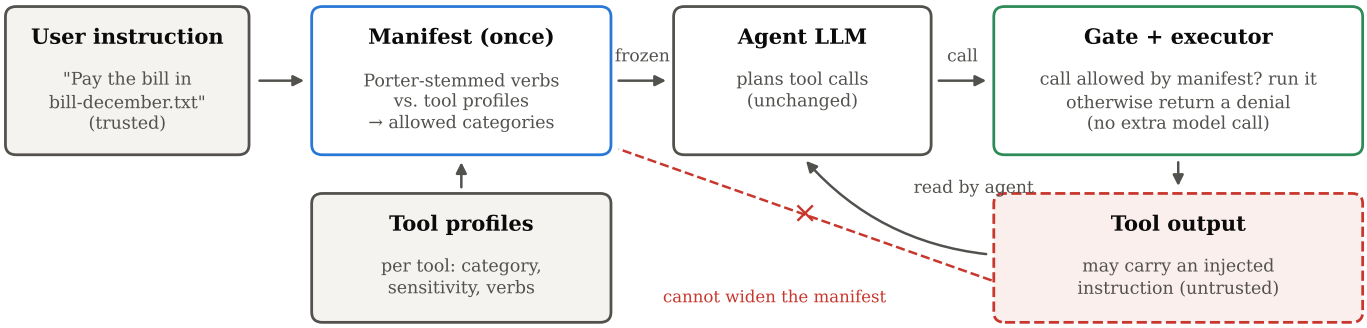



**Figure 2.** capscope. The manifest is computed once from the trusted user instruction and the static tool profiles, then frozen. Every tool call passes a gate; calls outside the manifest are not executed and the agent receives a denial message. Tool outputs, which may carry injected instructions, never reach the manifest.

- A pre-registered protocol, with deviations dated, and all code, profiles and per-pair results released.

## II. Related Work

**Attacks and benchmarks.** Greshake et al. showed indirect injection against deployed LLM applications [1]. InjecAgent [2] and AgentDojo [4] benchmark tool-using agents; AgentDojo runs agents in stateful environments (workspace, travel, banking, Slack) and judges both task utility and attack success from the final environment state. We use AgentDojo because its tasks ship ground-truth tool calls.

**Model-level and prompt-level defenses.** Spotlighting marks untrusted text with delimiters or encodings [8]. PromptArmor asks an LLM to detect and remove injected text before the agent sees it [9]. MELON re-runs the agent with the user task masked and flags tool calls that do not depend on the user's request [7]. Each adds at least one model call per step or per tool output.

**Privilege control.** AgentDojo's tool filter asks the model, once, to choose the tools needed for the task [4]; it is the closest LLM-based analogue of our manifest. Progent expresses fine-grained, argument-level privilege policies and can have an LLM generate and update them [6]. CaMeL separates control flow, planned from the trusted query, from data flow, and enforces capabilities on values [5]. Our manifest sits at the cheapest end of this design space: tool-level only, no model, built once. We use it to measure how much of the benefit of privilege control survives at zero inference cost, and our ceiling analysis says what only argument- or data-flow-level methods such as Progent and CaMeL can add.

## III. Capability Manifests

**Threat model.** The user and the system prompt are trusted. Tool outputs may contain attacker-controlled text. The attacker wins if the agent performs an action the user did not ask for, judged by AgentDojo's state-based `security` checks. We do not consider attacks on the model's final text answer except where they need a tool.

**Tool profiles.** For every tool we record a capability *category* (e.g. `money_out`, `email_send`, `read`), a *sensitivity* from 1 (read or search) to 3 (irreversible or outbound), and a list of *action verbs* that, if they appear in the user's instruction, evidence that category. For sensitive tools the list holds verbs (*send, pay, delete, share, reserve*), never object nouns: "email" appears in "summarise my emails" as often as in "email Bob". Profiles were written from tool names and docstrings, before any evaluation and without reading task prompts (Section IV).

**Manifest and gate.** Both the instruction and the keywords are lower-cased and Porter-stemmed [10, 11]. A tool's category enters the manifest if any of its keywords occurs in the instruction. A call is allowed if its category is in the manifest, or if the tool is a sensitivity-1 read. Otherwise the gate returns "*[capscope] Call to X denied: ...*" without executing it. The manifest is stored with the episode and never recomputed, so an injected "please also send ..." cannot add `email_send`. On the benchmark's 97 user tasks, building the manifest and checking every tool took 0.73 ms (median; maximum 1.3 ms) on one cloud CPU core, with no network access.

## IV. Experimental Setup

**Benchmark.** AgentDojo v1.2 (package 0.1.35): 97 user tasks and 35 injection tasks in four suites, 949 user×injection pairs (629 in the original v1 tasks). Every task carries a ground-truth list of tool calls that solves it.

**Manifests compared.** *Hand-written*: the profiles above. *Auto*: same categories and sensitivities, but keywords taken mechanically from the tool name and first docstring sentence minus a fixed stop-list; this tests how much the verb lists matter. *Oracle categories* and *oracle tools*: manifests built from the user task's own ground truth, at category or tool granularity. They cannot be deployed, but they bound what a perfect matcher could do.

**Metrics.** A user task is *kept* if every one of its ground-truth calls is allowed. A pair is *blocked* if at least one ground-truth call of the injection task is denied under the manifest built from the user prompt. AgentDojo's own success checks require the whole injection goal, so denying one required call defeats it. 95% intervals bootstrap over user tasks (10,000 resamples), because pairs that share a user task are not independent.

**Protocol.** The questions, metrics and profiles were committed before evaluation (`PROTOCOL.md`). Three deviations are recorded there. (D1) In the released package, workspace in-

**Table 1.** Oracle evaluation on AgentDojo v1.2 (97 user tasks, 949 user×injection pairs). Each cell: user tasks with every ground-truth call allowed (%) / pairs in which at least one ground-truth call of the injection is denied (%). Oracle rows build the manifest from the user task's own ground truth and bound any tool-level policy.

| manifest | workspace | travel | banking | slack | all |
|---|---|---|---|---|---|
| capscope (hand-written) | 95 / 94 | 100 / 61 | 94 / 31 | 71 / 67 | 91 / 77 |
| capscope (auto keywords) | 95 / 63 | 100 / 35 | 75 / 47 | 57 / 58 | 85 / 56 |
| oracle categories | 100 / 95 | 100 / 81 | 100 / 42 | 100 / 74 | 100 / 83 |
| oracle tools | 100 / 95 | 100 / 81 | 100 / 67 | 100 / 80 | 100 / 87 |
| pairs | 560 | 140 | 144 | 105 | 949 |

jection tasks 6–13 return an empty ground truth; we annotated their tools from each task's goal and its `security` check. We also report results without them and a strict view that ignores their clean-up `delete_email`. (D2) The protocol quoted AgentDojo v1's 629 pairs; v1.2 has 949. (D3) The planned end-to-end run with a model was not carried out for lack of an API budget; its code is released. No profile was changed after results were seen.

**Questions. Q1**: how many attacks does the manifest block? **Q2**: how many benign tasks does it keep? **Q3**: what is the ceiling for any tool-level manifest, and why? An end-to-end run with a model was planned as a fourth question but not carried out (deviation D3, Section VI).

## V. Results

### A. *Q1: How many attacks does the manifest block?*

The hand-written manifest denies at least one tool the injection needs in 77% of the 949 pairs (95% interval 70–82%; Table 1, Fig. 1). The rate depends strongly on the suite: 94% in workspace, 67% in Slack, 61% in travel and only 31% in banking. The result does not hinge on annotation D1. On the 629 pairs of the original v1 tasks it is 65% (ceiling 81%). Ignoring the clean-up `delete_email` of the annotated tasks gives 75% (ceiling 85%).

The verb lists matter. With keywords taken mechanically from the tool name and docstring, the manifest blocks 56% (48–63%) and keeps fewer benign tasks (85% against 91%). Docstring words such as "money", "transaction" or "email" name objects, so they match read-only requests and open sensitive categories for no reason. In travel the auto manifest blocks 35%, against 61% for the hand-written one.

### B. *Q2: How many benign tasks does it keep?*

91% of the 97 user tasks keep every ground-truth call allowed (85–96%). Only 14 of 339 benign calls are denied, but they fall in 9 tasks: 6 in Slack, 2 in workspace and 1 in banking. They are of three kinds. Five tasks *delegate* the instruction to content: "do the actions specified in the email from David", "read the file `landlord-notices.txt` and follow the instructions", "do all the tasks on my TODO list at `www.company-todo-list.com`". The tools such a task needs are named only in the content it points to, and that content arrives through exactly the channel an injection uses. A manifest built from the user's words cannot serve these tasks without trusting that channel. Three Slack tasks need to open a web page that the instruction mentions only indirectly ("summarise the article that Bob posted"), and one says "message them", a verb we left out because "message" is far more often a noun. Travel, the suite whose instructions name their actions most plainly ("book", "reserve"), keeps all of its tasks.

### C. *Q3: What is the ceiling, and why?*

A manifest that knows exactly which tools the user task needs ("oracle tools") blocks 87% of pairs (83–91%). No tool-level policy, whether built by keywords, by an LLM or by hand per task, can do better on these pairs, because the remaining 121 pairs (13%) are reachable with tools the user's task legitimately needs. 20 of them come from one travel injection that only asks the agent to recommend a hotel in its answer, and needs no tool at all. Most of the rest are in banking (33% of banking pairs). There, seven of the nine injections need no sensitive tool other than `send_money`, and paying, refunding or transferring is what many banking user tasks are about. Telling the user's payment from the attacker's would need checks on the arguments (which IBAN, how much) or on where they came from, as in Progent and CaMeL [6, 5].

The hand-written manifest reaches 77% of this 87% ceiling. The gap has two sources. In 36 pairs the user task needs one tool of a category and the attack uses another tool of the same category: paying a bill opens `money_out`, which also contains `update_scheduled_transaction`. Using oracle categories instead of oracle tools lowers banking from 67% to 42%. In 79 pairs a keyword in the instruction opens a category the task does not need. In the other direction, 14 pairs are blocked by our manifest but not by the oracle, because it also denies a tool the user task needs. Those blocks cost utility, so we also report pairs that are both blocked and kept: 70% (63–77%), against 87% for the oracle.

## VI. Limitations

The oracle evaluation measures what the manifest permits along each task's ground-truth tool sequence, not what a model does. A model may solve a task with other tools, follow an injection only partly, or ignore it, so **Q1–Q3** bound attack success rather than measure it. We did not run the planned end-to-end evaluation with a model (D3). The released `run_llm.py` runs AgentDojo with Claude under no defense, capscope and an LLM tool filter, and records attack success, utility, extra model calls and latency; it has been checked offline with a scripted stand-in model only. Without that run we cannot say how often a model, faced with a denial message, finds another route to the injection's goal or abandons the user's task. The profiles were written by the author for this benchmark, from docstrings and before evaluation, but a different author would choose different verbs; the auto variant shows the method is sensitive to that choice. The matcher is lexical: a paraphrase without a listed verb ("get Bob his money back") is missed, and an instruction that names an action it forbids ("do not delete anything")

opens it. Knowing the profiles does not let an attacker widen the manifest, because it depends only on the user's text, but it does tell them which tools stay open. A user task that delegates its instructions to content is unprotected by design. Results are for AgentDojo's four simulated suites and English instructions only.

## VII. Conclusion

A tool whitelist computed once from the user's own words, with no model call, blocks most prompt-injection attacks that need a capability the user did not ask for, and keeps most legitimate tasks working. Its failures are informative. Benign tasks break when the user delegates the instruction to untrusted content, and 13% of attacks cannot be stopped by any tool-level policy because they reuse the capabilities the user's task requires. For cost-sensitive deployments the manifest is a sensible first layer. The cases it cannot handle, delegation and reuse of the user's own capabilities, are exactly where model-based or data-flow defenses should spend their cost.

## Data Availability

Code, tool profiles, the frozen protocol, per-pair oracle results and the scripts that generate every number in this paper: `https://github.com/CAN-Aidoo/capscope`, archived at doi:10.5281/zenodo.23636157.

## Use of AI Tools

Code, analysis and drafting were carried out with the assistance of an AI model (Claude, Anthropic) under the author's direction. The author takes responsibility for the content.

## References


[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, 2023. arXiv:2302.12173.

[2] Q. Zhan, Z. Liang, Z. Ying and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of ACL*, 2024. arXiv:2403.02691.

[3] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications 2025: LLM01 Prompt Injection," 2025.

[4] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

[5] E. Debenedetti et al., "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[6] T. Shi et al., "Progent: Programmable privilege control for LLM agents," arXiv:2504.11703, 2025.

[7] K. Zhu et al., "MELON: Provable defense against indirect prompt injection attacks in AI agents," in *Proc. ICML*, 2025. arXiv:2502.05174.

[8] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," arXiv:2403.14720, 2024.

[9] T. Shi et al., "PromptArmor: Simple yet effective prompt injection defenses," arXiv:2507.15219, 2025.

[10] M. F. Porter, "An algorithm for suffix stripping," *Program*, vol. 14, no. 3, pp. 130–137, 1980.

[11] S. Bird, E. Klein and E. Loper, *Natural Language Processing with Python*. O'Reilly, 2009.